

Meshr-Lang

Documentation du langage

G. Prince - Architecte Principal

Version 0.2.0 - Septembre 2025

Contents

1	Meshr-Lang	7
1.1	Documentation du langage	7
2	1	8
3	Introduction	8
3.1	Contexte et motivation	8
3.2	Philosophie du langage	8
3.2.1	Déclaratif avant tout	8
3.2.2	Data Mesh Native	8
3.2.3	Pragmatique et évolutif	8
3.3	Valeur ajoutée	9
3.4	Public cible	9
3.5	Vision à long terme	9
4	2	10
5	Objectif du langage	10
5.1	Vision et philosophie	10
5.2	Rationnel du langage	10
5.2.1	Pourquoi un nouveau langage ?	10
5.2.2	Choix de design	11
5.2.3	Objectifs principaux	11
5.3	Avantages clés	12
6	3	13
7	Syntaxe de base	13
7.1	Structure des fichiers	13
7.2	Style déclaratif	13
7.3	Mots-clés réservés	13
7.3.1	Déclarations principales	13
7.3.2	Métriques (nouveau v0.2.0)	13
7.3.3	Relations et composition	14

7.3.4	Annotations et modificateurs	14
7.3.5	Types de base	14
7.3.6	Opérateurs et mots-clés logiques	14
7.3.7	Exemples d'usage incorrect	14
7.4	Commentaires	14
7.4.1	Commentaires ligne simple	14
7.4.2	Commentaires multi-lignes	15
7.5	Types de valeurs primitives	15
7.5.1	Types numériques	15
7.5.2	Types textuels	15
7.5.3	Types temporels	15
7.5.4	Types logiques	15
7.5.5	Types spécialisés	15
7.5.6	Types composés	16
7.6	Structure d'un module	16
7.6.1	Ordre des déclarations	16
8	4	18
9	Déclaration des modules et imports	18
9.1	Concept et objectifs	18
9.2	Architecture modulaire	18
9.2.1	Principes fondamentaux	18
9.2.2	Organisation hiérarchique	18
9.3	Syntaxe	18
9.3.1	Déclaration de module	18
9.3.2	Import simple	18
9.3.3	Import sélectif	19
9.3.4	Import avec alias	19
9.4	Sémantique	19
9.4.1	Résolution des noms	19
9.4.2	Gestion des conflits	19
9.5	Types d'imports supportés	20
9.5.1	Import de types de base	20
9.5.2	Import d'entités	20
9.5.3	Import d'énumérations	20
9.5.4	Import de traits	20
9.5.5	Import d'aspects	20
9.5.6	Import d'annotations	20
9.6	Ordre des déclarations	20
9.7	Export des déclarations	22
9.7.1	Export groupé (recommandé)	22
9.7.2	Export inline	22
9.7.3	Export sélectif avec annotations	22
9.8	Convention d'arborescence	22
9.8.1	Structure recommandée	22
9.8.2	Mapping nom de module ↔ fichier	23
9.8.3	Bonnes pratiques	23

10 5	24
11 Annotations	24
11.1 Syntaxe	24
11.1.1 Annotation simple	24
11.1.2 Annotation avec paramètre unique	24
11.1.3 Annotation avec paramètres nommés	24
11.1.4 Annotations multiples	24
11.2 Règles	25
11.2.1 Placement des annotations	25
11.2.2 Ordre des annotations	25
11.3 Annotations avec arguments typés	25
11.3.1 Types supportés dans les annotations	25
11.3.2 Exemples d'usage	25
11.4 Déclaration des types d'annotations	26
11.4.1 Annotation simple	26
11.4.2 Annotation complexe	26
11.5 Types de base autorisés pour les annotations	26
11.5.1 Types primitifs	26
11.5.2 Collections	27
11.5.3 Énumérations	27
11.6 Contraintes sur les types String	27
11.6.1 Patterns de validation	27
11.6.2 Contraintes de longueur	27
11.7 Annotations Standard (StdLib)	28
11.7.1 Annotations de versioning	28
11.7.2 Annotations de documentation	28
11.7.3 Annotations de gouvernance	28
11.7.4 Annotations de qualité	28
11.7.5 Annotations de scope	28
12 6	30
13 Énumérations	30
13.1 Concept et objectifs	30
13.2 Types d'énumérations	30
13.2.1 Énumérations simples (valeurs string)	30
13.2.2 Énumérations avec identifiants	30
13.2.3 Énumérations mixtes	30
13.3 Syntaxe simple (inline)	30
13.3.1 Déclaration basique	30
13.3.2 Avec annotations	31
13.3.3 Usage dans les entités	31
13.4 Export groupé	31
13.4.1 Export simple	31
13.4.2 Export avec annotations	31
13.5 Règles	31
13.5.1 Règles de nommage	31

13.5.2 Règles de validation	32
13.5.3 Exemples de bonnes pratiques	32
13.6 Enums avec attributs typés	32
13.6.1 Syntaxe avancée	32
13.6.2 Avec types complexes	32
13.6.3 Usage des énumérations avec attributs	33
13.6.4 Validation et contraintes	33
14 7	34
15 Records et Collections	34
15.1 Concept et objectifs	34
15.2 Syntaxe des records	34
15.2.1 Record simple	34
15.2.2 Record avec contraintes	34
15.2.3 Record avec annotations	34
15.3 Règles sémantiques	35
15.3.1 Différences avec les entités	35
15.3.2 Usage recommandé	35
15.3.3 Composition et imbrication	35
15.4 Collections typées	35
15.4.1 Types de collections supportés	35
15.4.2 Collections imbriquées	36
15.5 Valeurs par défaut	37
15.5.1 Dans les records	37
15.5.2 Dans les traits	37
15.5.3 Dans les aspects	37
15.5.4 Règles sémantiques	37
16 8	38
17 Entités et Relations	38
17.1 Entités	38
17.1.1 Concept et rôle	38
17.1.2 Caractéristiques des entités	38
17.1.3 Syntaxe de base	38
17.1.4 Aspects dans les entités	39
17.2 Relations	40
17.2.1 Types de relations	40
17.2.2 Relations avec aspects	41
17.2.3 Type Relations (Templates réutilisables)	41
17.2.4 Règles sémantiques	42
18 9	43
19 Traits	43
19.1 Concept et objectifs	43
19.1.1 Principe de composition	43
19.1.2 Avantages	43

19.2	Syntaxe de base	43
19.2.1	Définition d'un trait	43
19.2.2	Trait avec contraintes	43
19.2.3	Trait avec valeurs par défaut	44
19.3	Utilisation des traits	44
19.3.1	Composition simple	44
19.3.2	Composition multiple	44
19.3.3	Composition avec aspects	44
19.4	Traits spécialisés	45
19.4.1	Traits de sécurité	45
19.4.2	Traits de versioning	45
19.4.3	Traits de métadonnées	45
19.5	Règles de composition	45
19.5.1	Résolution des conflits	45
19.5.2	Héritage de contraintes	46
19.5.3	Traits avec annotations	46
20	10	47
21	Aspects	47
21.1	Concept et objectifs	47
21.1.1	Principe de séparation des préoccupations	47
21.1.2	Domaines d'application	47
21.2	Syntaxe de base	47
21.2.1	Définition d'un aspect	47
21.2.2	Aspect avec contraintes	47
21.2.3	Aspect avec énumérations	48
21.3	Application des aspects	48
21.3.1	Sur les entités	48
21.3.2	Sur les relations	49
21.3.3	Sur les métriques	49
21.4	Aspects prédéfinis (StdLib)	50
21.4.1	Gouvernance des données	50
21.4.2	Cycle de vie des données	50
21.4.3	Observabilité et monitoring	50
21.5	Règles sémantiques	51
21.5.1	Application multiple	51
21.5.2	Héritage via les traits	51
21.5.3	Validation et cohérence	52
22	11	53
23	Métriques (Metrics)	53
23.1	Introduction	53
23.1.1	Philosophie et objectifs	53
23.2	Syntaxe de base	53
23.2.1	Structure générale	53
23.2.2	Exemple simple	53

23.2.3	Exemple complexe	54
23.3	Composants détaillés	54
23.3.1	1. Source (obligatoire)	54
23.3.2	2. Calcul (obligatoire)	55
23.3.3	3. Aggrégation (optionnel)	55
23.3.4	4. Unité (optionnel)	55
23.3.5	5. Outputs (optionnel)	56
23.3.6	6. Dimensions (optionnel)	56
23.3.7	7. Filtrage (optionnel)	57
23.3.8	8. Temporal (optionnel)	57
23.3.9	9. Aspects (optionnel)	57
23.4	Pattern Matching avancé	58
23.4.1	Syntaxe du pattern matching	58
23.4.2	Patterns supportés	58
23.5	Exemple complet	59
23.6	Règles sémantiques	61
23.6.1	Validation des références	61
23.6.2	Cohérence temporelle	61
23.6.3	Types et contraintes	62
23.6.4	Génération d'artefacts	62
24	12	63
25	Modificateur sealed	63
25.1	Concept et objectifs	63
25.1.1	Principe d'extensibilité contrôlée	63
25.1.2	Cas d'usage	63
25.2	Syntaxe	63
25.2.1	Énumération sealed	63
25.2.2	Trait sealed	63
25.2.3	Entity sealed	63
25.3	Règles sémantiques	64
25.3.1	Restrictions d'extension	64
25.3.2	Garanties de stabilité	64
25.3.3	Exemples d'usage	64
26	13	66
27	Artefacts définissables	66
27.1	Types d'artefacts supportés	66
27.1.1	Artefacts de structure	66
27.1.2	Artefacts de composition	66
27.1.3	Artefacts de métadonnées	66
27.1.4	Artefacts analytiques	66
27.2	Règles de définition	66
27.2.1	Nommage et conventions	66
27.2.2	Ordre de déclaration	67
27.2.3	Visibilité et encapsulation	67

Logo Meshr-Lang

Figure 1: Logo Meshr-Lang

27.3 Exemples d'architecture complète	67
27.3.1 Module de domaine métier	67
28 Annexes	70
28.1 Annexe A — Grammaire EBNF (référence)	70
28.1.1 Structure générale	70
28.1.2 Métriques (v0.2.0)	70
28.1.3 Types et contraintes	71
28.2 Annexe B — Grammaire ANTLR (référence, Python target)	71
28.2.1 Tokens principaux (v0.2.0)	71
28.2.2 Règles de parsing	72

1 Meshr-Lang

1.1 Documentation du langage

Version 0.2.0 - Septembre 2025

Avec support complet des métriques et pattern matching

G. Prince - Architecte Principal

2025-09-18

2 1

3 Introduction

Meshr-Lang est un langage de modélisation déclaratif conçu spécifiquement pour l'architecture Data Mesh et la gouvernance des données à l'échelle de l'entreprise. Il répond aux défis modernes de la gestion des données distribuées en offrant une syntaxe claire, expressive et alignée sur les principes de l'architecture Data Mesh.

3.1 Contexte et motivation

Dans un monde où les données deviennent le moteur de l'innovation, les organisations font face à des défis croissants :

- Explosion du volume de données : Multiplication des sources et formats
- Complexité organisationnelle : Données dispersées dans des silos métier
- Exigences de conformité : RGPD, SOX, HIPAA et autres réglementations
- Besoin d'agilité : Time-to-market critique pour les produits data
- Qualité et fiabilité : Confiance dans les données pour la prise de décision

L'architecture Data Mesh émerge comme une solution à ces défis, mais nécessite des outils et langages adaptés pour être mise en œuvre efficacement.

3.2 Philosophie du langage

Meshr-Lang s'inspire de plusieurs principes fondamentaux :

3.2.1 Déclaratif avant tout

- Ce que vous voulez, pas comment l'obtenir
- Focus sur la sémantique métier plutôt que l'implémentation technique
- Lisibilité et expressivité au cœur du design

3.2.2 Data Mesh Native

- Domaine-centré : Chaque module représente un domaine métier
- Décentralisé : Pas de dépendance à une architecture centralisée
- Self-serve : Outils et patterns intégrés pour l'autonomie des équipes

3.2.3 Pragmatique et évolutif

- Syntaxe simple mais puissante
- Extensibilité via les annotations et aspects

- Intégration avec l'écosystème existant

3.3 Valeur ajoutée

Meshr-Lang apporte une valeur unique dans l'écosystème des langages de modélisation :

Aspect	Meshr-Lang	Alternatives
Focus Data Mesh	Natif	Générique
Gouvernance intégrée	Built-in	Externe
Syntaxe déclarative	Simple	Complexe
Annotations riches	Expressives	Basiques
Évolutivité	Modulaire	Monolithique

3.4 Public cible

Ce langage s'adresse à :

- Data Architects : Conception d'architectures Data Mesh
- Data Product Owners : Définition de produits de données
- Data Stewards : Gouvernance et qualité des données
- Data Engineers : Implémentation et intégration
- Data Analysts : Compréhension et utilisation des données
- Compliance Officers : Conformité et audit

3.5 Vision à long terme

Meshr-Lang aspire à devenir le standard de facto pour la modélisation Data Mesh, en offrant :

- Adoption large dans l'industrie
- Écosystème riche d'outils et intégrations
- Communauté active de contributeurs
- Formation et certification pour les professionnels

4 2

5 Objectif du langage

Meshr-Lang est un langage déclaratif spécialisé conçu pour décrire et modéliser les artefacts d'une architecture Data-as-a-Product et d'entreprise. Il permet de définir de manière structurée et exécutable les domaines métier, produits de données, contrats, aspects (métadonnées), équipes, politiques, et leurs interrelations.

5.1 Vision et philosophie

Meshr-Lang s'inscrit dans la philosophie du Data Mesh et des architectures d'entreprise modernes où :

- La donnée est un produit : Chaque dataset est traité comme un produit avec ses propres propriétaires, contrats, et SLA
- La documentation est vivante : Le code source devient la source de vérité pour l'architecture
- L'automatisation est centrale : Les modèles déclaratifs génèrent automatiquement les artefacts techniques
- La gouvernance est intégrée : Les politiques et contraintes sont exprimées directement dans le modèle

5.2 Rationnel du langage

5.2.1 Pourquoi un nouveau langage ?

L'écosystème actuel des langages de modélisation présente plusieurs limitations pour l'architecture Data Mesh :

Langages existants inadaptés : - UML : Trop générique, pas de support natif pour les concepts Data Mesh - JSON Schema : Syntaxe verbeuse, pas de support pour les relations complexes - YAML : Pas de validation de types, syntaxe peu expressive - GraphQL Schema : Limité aux APIs, pas de support pour la gouvernance - OpenAPI : Focus API uniquement, pas de modélisation métier

Besoins spécifiques Data Mesh : - Domaine-centré : Modélisation des domaines métier et leurs frontières - Gouvernance intégrée : Support natif pour les politiques et contraintes - Relations complexes : Modélisation des dépendances entre produits de données - Métadonnées riches : Annotations et aspects pour la documentation - Évolutivité : Support pour l'architecture distribuée et décentralisée

5.2.2 Choix de design

1. Syntaxe déclarative

```
// Ce que vous voulez , pas comment l'obtenir
entity Customer is
  id : String
  name : String
  email : String pattern "^[^@]+@[^@]+$"
end
```

2. Types riches et constraints

```
// Types avec contraintes intégrées
type Email is String pattern "^[^@]+@[^@]+$"
type Age is Integer range 0..150
type PhoneNumber is String pattern "^\+?[1-9]\d{1,14}$"
```

3. Annotations expressives

```
@Version("1.2.0")
@author(name="Data Team", email="data@company.com")
@Documented(summary="Customer data model")
@Confidentiality("internal")
entity Customer is
  // ...
end
```

4. Relations sémantiques

```
// Relations avec sémantique métier
relation CustomerOrder from Customer(id) to Order(customerId)
relation ProductCategory from Product(id) to Category(id)
```

5.2.3 Objectifs principaux

Meshr-Lang vise à résoudre les défis concrets de l'architecture Data Mesh :

- 1. Modélisation déclarative** - Définition claire des domaines métier et de leurs frontières - Spécification des contrats entre domaines - Documentation vivante et exécutable
- 2. Gouvernance intégrée** - Politiques de sécurité et de conformité exprimées dans le modèle - Traçabilité et audit automatiques - Gestion des versions et de la compatibilité
- 3. Génération d'artefacts** - Schémas de bases de données - APIs REST et GraphQL - Documentation technique - Pipelines de données

4. Validation et vérification - Cohérence des modèles - Respect des contraintes métier - Détection de dépendances circulaires

5.3 Avantages clés

Productivité - Réduction du temps de développement grâce à la génération automatique - Réutilisation des composants via les traits et aspects - Documentation automatique et toujours à jour

Qualité - Validation statique des modèles - Cohérence garantie entre les artefacts - Tests automatiques des contrats

Évolutivité - Architecture modulaire permettant l'ajout de nouveaux concepts - Intégration : Compatible avec l'écosystème moderne (CI/CD, IaC, observabilité) - Validation : Vérification statique des modèles et détection d'erreurs précoces

7 Syntaxe de base

Meshr-lang est un langage de modélisation déclaratif conçu pour décrire des architectures d'entreprise, des produits, des domaines métier et leurs relations. Sa syntaxe s'inspire de langages modernes comme HCL (HashiCorp Configuration Language), Kotlin DSL, et GraphQL Schema Definition Language (SDL).

7.1 Structure des fichiers

Extension de fichier : .meshr

Tous les fichiers Meshr-lang utilisent l'extension .meshr et suivent une structure modulaire où chaque fichier représente un module autonome contenant des déclarations de types, d'entités, d'énumérations, et d'autres artefacts métier.

7.2 Style déclaratif

Meshr-lang adopte un style déclaratif à blocs qui privilégie la lisibilité et la structure hiérarchique :

- Blocs imbriqués : Les déclarations sont organisées en blocs logiques avec indentation
- Syntaxe claire : Utilisation de mots-clés explicites et de ponctuation minimale
- Lisibilité : Structure qui reflète naturellement la hiérarchie des concepts métier

7.3 Mots-clés réservés

Les mots suivants sont réservés par le langage et ne peuvent pas être utilisés comme identifiants de champs, variables ou noms d'entités :

7.3.1 Déclarations principales

- module, import, export
- entity, enum, trait, aspect, annotation, record, relation
- type, sealed, bidirectional

7.3.2 Métriques (nouveau v0.2.0)

- metric, source, calculation, aggregation, unit
- outputs, dimensions, filters, temporal
- window, refresh_frequency, historical_depth
- match, or (pattern matching)

7.3.3 Relations et composition

- from, to, with, extends, implements
- of, is, end, aspects

7.3.4 Annotations et modificateurs

- required, optional, default
- pattern, range, length, precision

7.3.5 Types de base

- String, Integer, Float, Boolean, Date, Timestamp, Interval
- List, Map, Range, Bytes, Geography, JSON, SQL

7.3.6 Opérateurs et mots-clés logiques

- and, or, not, in, contains
- true, false, null

7.3.7 Exemples d'usage incorrect

```
// □ INCORRECT - Utilisation de mots-clés réservés
entity module is // 'module' est réservé
  type : String  // 'type' est réservé
end
```

```
// □ CORRECT - Utilisation d'identifiants valides
entity DataModule is
  data_type : String
end
```

7.4 Commentaires

Meshr-lang supporte deux types de commentaires :

7.4.1 Commentaires ligne simple

```
// Ceci est un commentaire ligne simple
entity Customer is
  id : String // Identifiant unique du client
end
```

7.4.2 Commentaires multi-lignes

```
/*  
 * Ceci est un commentaire multi-lignes  
 * Utile pour des descriptions détaillées  
 * ou de la documentation  
 */  
entity Customer is  
    id : String  
end
```

7.5 Types de valeurs primitives

Meshr-lang fournit un ensemble riche de types primitifs optimisés pour la modélisation de données :

7.5.1 Types numériques

- Integer : Nombres entiers (-2^{63} à $2^{63}-1$)
- Float : Nombres décimaux (IEEE 754 double precision)

7.5.2 Types textuels

- String : Chaînes de caractères Unicode
- Bytes : Données binaires

7.5.3 Types temporels

- Date : Date sans heure (YYYY-MM-DD)
- Timestamp : Date et heure avec timezone
- Interval : Durée ou période

7.5.4 Types logiques

- Boolean : true ou false

7.5.5 Types spécialisés

- Geography : Coordonnées géographiques (GeoJSON)
- JSON : Structures JSON natives
- SQL : Requêtes SQL typées

7.5.6 Types composés

- List of T : Liste typée d'éléments
- Map of K to V : Dictionnaire typé clé-valeur
- Range of T : Plage de valeurs

7.6 Structure d'un module

Un module Meshr-lang suit une structure standardisée :

```
// 1. Déclaration du module
@Version("1.0.0")
@Author(name="Team", email="team@company.com")
module domain.subdomain

// 2. Imports (optionnel)
import { CommonTypes } from shared.types
import { SecurityAspects } from meshr.security

// 3. Exports (optionnel)
export { Customer, Order, CustomerOrder }

// 4. Déclarations locales
enum Status is ("active", "inactive")

entity Customer is
  id : String
  name : String
  status : Status
end

// 5. Relations
relation CustomerOrder from Customer(id) to Order(customer_id)
```

7.6.1 Ordre des déclarations

1. Annotations du module
2. Déclaration module
3. Déclarations import
4. Déclarations export
5. Types, énumérations, traits, aspects

- 6. Entités
- 7. Relations
- 8. Métriques

8 4

9 Déclaration des modules et imports

9.1 Concept et objectifs

Le système de modules de Meshr-Lang est conçu pour supporter une architecture décentralisée et modulaire, alignée sur les principes du Data Mesh. Chaque module représente un domaine métier autonome avec ses propres types, entités, et règles métier.

9.2 Architecture modulaire

9.2.1 Principes fondamentaux

- **Autonomie** : Chaque module est indépendant et auto-suffisant
- **Encapsulation** : Les détails internes sont cachés, seules les interfaces publiques sont exposées
- **Réutilisabilité** : Les modules peuvent être importés et réutilisés dans d'autres contextes
- **Évolutivité** : Les modules peuvent évoluer indépendamment sans casser les dépendances

9.2.2 Organisation hiérarchique

```
company.domain.subdomain |—
  shared.types             // Types communs |—
  customer.core            // Domaine client |—
  customer.analytics       // Analytics client |—
  order.management        // Gestion des commandes |—
  billing.invoicing        // Facturation
```

9.3 Syntaxe

9.3.1 Déclaration de module

```
@Version("1.2.0")
@Author(name="Customer Team", email="customer@company.com")
@Documented(summary="Customer domain core entities")
module customer.core
```

9.3.2 Import simple

```
// Import d'un module complet
import shared.types
```

```
// Utilisation
entity Customer is
  id : shared.types.ID
  email : shared.types.Email
end
```

9.3.3 Import sélectif

```
// Import de déclarations spécifiques
import { ID, Email, Address } from shared.types
import { DataClassification } from meshr.security

// Utilisation directe
entity Customer is
  id : ID
  email : Email
  address : Address
end
```

9.3.4 Import avec alias

```
// Import avec renommage
import { Customer as BaseCustomer } from shared.entities
import { DataClassification as DC } from meshr.security

entity EnhancedCustomer is
  base : BaseCustomer
  classification : DC
end
```

9.4 Sémantique

9.4.1 Résolution des noms

1. **Scope local** : Déclarations dans le module courant
2. **Imports explicites** : Déclarations importées
3. **Stdlib** : Bibliothèque standard Meshr-Lang

9.4.2 Gestion des conflits

```
// En cas de conflit , utiliser le nom qualifié
import { Customer } from domain.a
import { Customer } from domain.b

entity Order is
  customer_a : domain.a.Customer
  customer_b : domain.b.Customer
end
```

9.5 Types d'imports supportés

9.5.1 Import de types de base

```
import { String , Integer , Date } from meshr.core
```

9.5.2 Import d'entités

```
import { Customer , Product } from commerce.entities
```

9.5.3 Import d'énumérations

```
import { Status , Priority } from shared.enums
```

9.5.4 Import de traits

```
import { WithTimestamps , WithSecurity } from meshr.traits
```

9.5.5 Import d'aspects

```
import { DataClassification , Audit } from meshr.aspects
```

9.5.6 Import d'annotations

```
import { BusinessCritical , Deprecated } from meshr.annotations
```

9.6 Ordre des déclarations

L'ordre des déclarations dans un module doit respecter la séquence suivante :

```

// 1. Annotations du module (obligatoire)
@Version("1.0.0")
module domain.name

// 2. Imports (optionnel)
import { Types } from other.module

// 3. Exports (optionnel)
export { Entity1 , Entity2 }

// 4. Déclarations de types et énumérations
enum Status is ("active", "inactive")
type Email is String pattern "^[^@]+@[^@]+$"

// 5. Annotations personnalisées
annotation BusinessRule is
  description : String
end

// 6. Traits
trait WithMetadata is
  created_at : Timestamp
  updated_at : Timestamp
end

// 7. Aspects
aspect Audit is
  auditor : String
  audit_date : Date
end

// 8. Entités
entity Customer is
  // ...
end

// 9. Relations
relation CustomerOrder from Customer(id) to Order(customer_id)

// 10. Métriques

```

```
metric CustomerSatisfaction is
  source Customer
  calculation avg(satisfaction_score)
end
```

9.7 Export des déclarations

9.7.1 Export groupé (recommandé)

```
// Export en début de module pour la lisibilité
export {
  Customer,
  Order,
  CustomerOrder,
  Status
}
```

9.7.2 Export inline

```
// Export lors de la déclaration
export entity Customer is
  id : String
  name : String
end

export enum Status is ("active", "inactive")
```

9.7.3 Export sélectif avec annotations

```
export {
  @Public Customer,           // Export public
  @Internal OrderMetrics,     // Export interne uniquement
  @Deprecated LegacyStatus    // Export avec avertissement
}
```

9.8 Convention d'arborescence

9.8.1 Structure recommandée

```
src/|—
  shared/| |—
```

```

types.meshr           // Types communs| ─
enums.meshr           // Énumérations partagées| ─
traits.meshr          // Traits réutilisables ─
customer/| ─
  core.meshr           // Entités principales| ─
  analytics.meshr      // Métriques et KPIs| ─
  events.meshr         // Événements métier ─
order/| ─
  management.meshr     // Gestion des commandes| ─
  fulfillment.meshr    // Exécution| ─
  billing.meshr        // Facturation ─
stdlib/ ─
  meshr/| ─
    core/              // Types de base| ─
    security/          // Aspects sécurité| ─
    governance/        // Gouvernance ─
  plugins/ ─
    gcp/               // Intégrations GCP

```

9.8.2 Mapping nom de module ↔ fichier

- customer.core → src/customer/core.meshr
- shared.types → src/shared/types.meshr
- meshr.security → stdlib / meshr/security/security .meshr

9.8.3 Bonnes pratiques

- **Un module = un fichier** : Correspondance 1:1
- **Noms explicites** : customer.analytics plutôt que customer.data
- **Hiérarchie logique** : Refléter l'organisation métier
- **Éviter la profondeur excessive** : Maximum 3-4 niveaux

10 5

11 Annotations

Les annotations en Meshr-Lang sont un mécanisme puissant pour attacher des métadonnées aux déclarations. Elles permettent d'enrichir le modèle avec des informations de gouvernance, de documentation, de validation, et d'intégration.

11.1 Syntaxe

11.1.1 Annotation simple

```
@Deprecated
entity OldCustomer is
  id : String
end
```

11.1.2 Annotation avec paramètre unique

```
@Version("2.1.0")
entity Customer is
  id : String
end
```

11.1.3 Annotation avec paramètres nommés

```
@Author(name="Data Team", email="data@company.com")
@Documented(
  summary="Customer entity with personal information",
  description="Represents a customer with GDPR compliance requirements"
)
entity Customer is
  id : String
  name : String
end
```

11.1.4 Annotations multiples

```
@Version("1.0.0")
@Author(name="Customer Team", email="customer@company.com")
@BusinessCritical(impact="high")
```

```

@Confidentiality("internal")
entity Customer is
  id : String
  personal_data : String
end

```

11.2 Règles

11.2.1 Placement des annotations

- **Modules** : Avant la déclaration module
- **Entités** : Avant la déclaration entity
- **Champs** : Avant la déclaration du champ
- **Relations** : Avant la déclaration relation
- **Métriques** : Avant la déclaration metric

11.2.2 Ordre des annotations

```

// Ordre recommandé : Version → Auteur → Documentation → Métadonnées métier
@Version("1.0.0")
@author(name="Team", email="team@company.com")
@Documented(summary="Entity description")
@BusinessCritical(impact="high")
entity Customer is
  // ...
end

```

11.3 Annotations avec arguments typés

11.3.1 Types supportés dans les annotations

- String : Texte libre
- Integer : Nombres entiers
- Float : Nombres décimaux
- Boolean : true ou false
- List of T : Listes typées
- Map of String to T : Dictionnaires

11.3.2 Exemples d'usage

```

@Stakeholders(
  owners=List["data.team@company.com", "business.owner@company.com"],

```

```

    reviewers=List ["architect@company.com"],
    approval_required=true,
    review_frequency=90
)
@DataClassification(
    level="confidential",
    encryption_required=true,
    retention_days=2555 // 7 ans
)
entity Customer is
    id : String
    personal_data : String
end

```

11.4 Déclaration des types d'annotations

11.4.1 Annotation simple

```

annotation Deprecated is
    reason : String = "No longer maintained"
    since : String
    replacement : String
end

```

11.4.2 Annotation complexe

```

annotation DataClassification is
    required level : String // "public", "internal", "confidential", "restricted"
    encryption_required : Boolean = false
    access_control : String = "role-based"
    retention_days : Integer
    steward : String

    // Validation des valeurs
    level pattern "^(public|internal|confidential|restricted)$"
    steward pattern "^[^@]+@[^@]+$" // Format email
end

```

11.5 Types de base autorisés pour les annotations

11.5.1 Types primitifs

```
annotation Metadata is
  title : String
  version : Integer
  active : Boolean
  score : Float
end
```

11.5.2 Collections

```
annotation Stakeholders is
  owners : List of String
  tags : List of String
  config : Map of String to String
end
```

11.5.3 Énumérations

```
enum Priority is ("low", "medium", "high", "critical")
```

```
annotation TaskInfo is
  priority : Priority
  categories : List of Priority
end
```

11.6 Contraintes sur les types String

11.6.1 Patterns de validation

```
annotation Contact is
  email : String pattern "^[^@]+@[^@]+$"
  phone : String pattern "^\\+?[1-9]\\d{1,14}$"
  url : String pattern "^https?:/*.*"
end
```

11.6.2 Contraintes de longueur

```
annotation Documentation is
  summary : String length 1..100
  description : String length 0..1000
  keywords : List of String length 1..50
end
```

11.7 Annotations Standard (StdLib)

11.7.1 Annotations de versioning

```
@Version("1.2.0")           // Version de la déclaration
@Since("1.0.0")             // Première version d'introduction
@Deprecated(
    since="2.0.0",
    reason="Replaced by NewEntity",
    replacement="domain.entities.NewEntity"
)
```

11.7.2 Annotations de documentation

```
@Documented(
    summary="Brief description",
    description="Detailed explanation of the entity purpose and usage",
    examples="Usage examples or references"
)
@author(name="Team Name", email="team@company.com")
```

11.7.3 Annotations de gouvernance

```
@BusinessCritical(
    impact="high",           // "low", "medium", "high", "critical"
    stakeholders=List["owner@company.com"]
)
@DataClassification(
    level="confidential",
    encryption_required=true,
    steward="data.steward@company.com"
)
```

11.7.4 Annotations de qualité

```
@Experimental           // Fonctionnalité en cours de développement
@Stable                 // API stable, production-ready
@Beta                   // Fonctionnalité en test, peut changer
```

11.7.5 Annotations de scope

```
@Scope(  
    visibility="internal",    // "public", "internal", "private"  
    domains=List["customer", "analytics"]  
)  
@Confidentiality("internal") // Niveau de confidentialité
```

12 6

13 Énumérations

13.1 Concept et objectifs

Les énumérations en Meshr-Lang permettent de définir des ensembles finis de valeurs nommées. Elles sont essentielles pour modéliser les domaines métier où certains champs ne peuvent prendre qu'un nombre limité de valeurs prédéfinies.

13.2 Types d'énumérations

13.2.1 Énumérations simples (valeurs string)

```
enum Status is ("active", "inactive", "pending", "archived")
enum Priority is ("low", "medium", "high", "critical")
```

13.2.2 Énumérations avec identifiants

```
enum OrderStatus is (
    draft ,
    submitted ,
    processing ,
    shipped ,
    delivered ,
    cancelled
)
```

13.2.3 Énumérations mixtes

```
enum PaymentMethod is (
    cash ,
    credit_card ,
    debit_card ,
    "bank_transfer", // Mot-clé réservé nécessite des guillemets
    paypal ,
    cryptocurrency
)
```

13.3 Syntaxe simple (inline)

13.3.1 Déclaration basique

```
enum Color is ("red", "green", "blue", "yellow")
```

13.3.2 Avec annotations

```
@Version("1.0.0")
@Documented(summary="Available product categories")
enum Category is ("electronics", "clothing", "books", "home")
```

13.3.3 Usage dans les entités

```
entity Product is
    id : String
    name : String
    category : Category
    status : Status
end
```

13.4 Export groupé

13.4.1 Export simple

```
export { Status , Priority , Category }
```

13.4.2 Export avec annotations

```
export {
    @Public Status ,
    @Internal DebugLevel ,
    @Deprecated OldStatus
}
```

13.5 Règles

13.5.1 Règles de nommage

- **Noms d'énumérations** : PascalCase (OrderStatus, PaymentMethod)
- **Valeurs d'énumérations** : snake_case ou strings (draft, "bank_transfer")
- **Cohérence** : Utiliser le même style dans tout le module

13.5.2 Règles de validation

- **Unicité** : Chaque valeur doit être unique dans l'énumération
- **Non-vide** : Une énumération doit contenir au moins une valeur
- **Immutabilité** : Les valeurs d'énumération ne peuvent pas être modifiées

13.5.3 Exemples de bonnes pratiques

// □ CORRECT

```
enum UserRole is ("admin", "moderator", "user", "guest")
enum HttpStatus is (ok, created, not_found, server_error)
```

// □ INCORRECT

```
enum MixedCase is ("Admin", "user", "GUEST") // Styles incohérents
enum Empty is () // Énumération vide
```

13.6 Enums avec attributs typés

13.6.1 Syntaxe avancée

```
enum HttpStatus is
  ok(code: 200, message: "OK"),
  created(code: 201, message: "Created"),
  not_found(code: 404, message: "Not Found"),
  server_error(code: 500, message: "Internal Server Error")
end
```

13.6.2 Avec types complexes

```
enum Region is
  europe(
    code: "EU",
    timezone: "CET",
    currencies: List["EUR", "GBP", "CHF"],
    languages: List["en", "fr", "de", "es"]
  ),
  north_america(
    code: "NA",
    timezone: "EST",
    currencies: List["USD", "CAD"],
    languages: List["en", "fr", "es"]
  ),
```

```

    asia_pacific(
        code: "AP",
        timezone: "JST",
        currencies: List["JPY", "AUD", "SGD"],
        languages: List["en", "ja", "zh", "ko"]
    )
end

```

13.6.3 Usage des énumérations avec attributs

```

entity Order is
    id : String
    region : Region
    status : HttpStatus

    // Accès aux attributs de l'énumération
    region_code : String = region.code
    status_message : String = status.message
end

```

13.6.4 Validation et contraintes

```

enum ProductCategory is
    electronics(
        tax_rate: Float range 0.0..1.0,
        warranty_months: Integer range 12..60,
        requires_certification: Boolean
    ),
    clothing(
        tax_rate: Float range 0.0..1.0,
        seasonal: Boolean,
        size_chart: String pattern "^(XS|S|M|L|XL|XXL)$"
    )
end

```

14 7

15 Records et Collections

15.1 Concept et objectifs

Les records en Meshr-Lang permettent de définir des structures de données légères et composées, similaires aux structs ou aux classes de données. Ils sont particulièrement utiles pour regrouper des données connexes sans la complexité des entités complètes.

15.2 Syntaxe des records

15.2.1 Record simple

```
record Address is
  street : String
  city : String
  postal_code : String
  country : String
end
```

15.2.2 Record avec contraintes

```
record Coordinates is
  latitude : Float range -90.0..90.0
  longitude : Float range -180.0..180.0
  altitude : Float = 0.0
end
```

15.2.3 Record avec annotations

```
@Version("1.0.0")
@Documented(summary="Contact information structure")
record ContactInfo is
  email : String pattern "^[^@]+@[^@]+$"
  phone : String pattern "^\\+?[1-9]\\d{1,14}$"
  preferred_contact : String = "email"
end
```

15.3 Règles sémantiques

15.3.1 Différences avec les entités

- **Records** : Structures de données simples, pas d'aspects ni de relations
- **Entités** : Objets métier complets avec identité, aspects, et relations

15.3.2 Usage recommandé

// □ CORRECT – Record pour structure de données

```
record Money is
  amount : Float
  currency : String
end
```

// □ CORRECT – Entité pour objet métier

```
entity Customer is
  id : String
  name : String
  address : Address // Utilise le record Address
  contact : ContactInfo // Utilise le record ContactInfo
end
```

15.3.3 Composition et imbrication

```
record PersonalInfo is
  first_name : String
  last_name : String
  birth_date : Date
  address : Address // Record imbriqué
end
```

```
entity Employee is
  employee_id : String
  personal_info : PersonalInfo
  hire_date : Date
end
```

15.4 Collections typées

15.4.1 Types de collections supportés

```

record Product is
  id : String
  name : String
  tags : List of String
  categories : List of Category // Référence à une énumération
  related_products : List of String // IDs des produits liés
end

```

```

record Configuration is
  name : String
  settings : Map of String to String
  feature_flags : Map of String to Boolean
  thresholds : Map of String to Float
end

```

```

record PriceRange is
  min_price : Float
  max_price : Float
  valid_range : Range of Float // Plage calculée
end

```

```

record AgeGroup is
  name : String
  age_range : Range of Integer // Ex: 18..65
end

```

15.4.2 Collections imbriquées

```

record Analytics is
  metrics : Map of String to List of Float
  dimensions : List of Map of String to String
  time_series : Map of String to List of Timestamp
end

```

15.5 Valeurs par défaut

15.5.1 Dans les records

```
record UserPreferences is
  language : String = "en"
  timezone : String = "UTC"
  notifications : Boolean = true
  theme : String = "light"
  tags : List of String = List[] // Liste vide par défaut
end
```

15.5.2 Dans les traits

```
trait WithMetadata is
  tags : List of String = List["default"]
  config : Map of String to String = Map{}
  limits : Range of Integer = Range 0..100
end
```

15.5.3 Dans les aspects

```
aspect DataQuality is
  metrics : List of String = List["accuracy", "completeness"]
  thresholds : Map of String to Float = Map{"accuracy": 0.95}
  score_range : Range of Integer = Range 0..100
end
```

15.5.4 Règles sémantiques

- Les collections sont **typées** : List of String, Map of String to Integer
- Les valeurs par défaut utilisent les **littéraux** : List ["a", "b"], Map{"k": "v"}
- Les collections peuvent être **imbriquées** : List of Map of String to String
- Les **références circulaires** sont interdites
- Les types dans les collections doivent être **définis** ou **importés**

16 8

17 Entités et Relations

Les Entités et Relations sont des artefacts déclaratifs pour la modélisation de données dans l'architecture Data-as-a-Product. Elles permettent de définir des structures de données persistantes et leurs interconnexions.

17.1 Entités

Une entité est un artefact fondamental de Meshr-lang qui représente une structure de données persistante dans l'architecture d'entreprise. Elle modélise les concepts métier centraux et leurs propriétés.

17.1.1 Concept et rôle

Les entités servent à :

- Modéliser les concepts métier : Représenter les objets centraux du domaine (Customer, Product, Order, etc.)
- Définir la structure des données : Spécifier les champs, types, et contraintes
- Assurer la cohérence : Fournir un modèle de données unifié à travers l'organisation
- Faciliter l'intégration : Servir de contrat pour les APIs et les échanges de données
- Documenter le domaine : Exprimer explicitement les concepts et leurs relations

17.1.2 Caractéristiques des entités

Une entité Meshr-lang possède :

- Champs typés : Chaque champ a un type explicite (String, Integer, Date, etc.)
- Contraintes : Validation des données via des patterns, ranges, ou contraintes personnalisées
- Aspects : Métadonnées structurées pour la gouvernance, sécurité, qualité
- Traits : Composition de comportements réutilisables
- Relations : Connexions sémantiques avec d'autres entités

17.1.3 Syntaxe de base

```
entity Customer is
  id : String
  name : String
  email : String
  created_at : Timestamp
end
```

```

entity Product is
  id : String
  name : String length 1..100
  price : Float range 0.01..999999.99
  sku : String pattern "^[A-Z]{2}\\d{6}$"
  category : ProductCategory
end

```

```

entity Order with WithTimestamps, WithSecurity is
  id : String
  customer_id : String
  total_amount : Float
  status : OrderStatus
end

```

17.1.4 Aspects dans les entités

```

entity Customer is
  id : String
  name : String
  email : String

  aspects {
    DataClassification {
      level: "confidential",
      encryption_required: true,
      access_control: "role-based",
      data_retention: Interval 7 year,
      steward: "privacy@company.com"
    },
    DataStewardship {
      steward: "customer.team@company.com",
      owner: "customer.domain@company.com",
      business_domain: "customer_management",
      last_review: "2024-01-01",
    }
  }
end

```

```
        next_review : "2024-12-31"
    }
}
end
```

17.2 Relations

Les relations définissent les connexions sémantiques entre entités, capturant les associations métier importantes.

17.2.1 Types de relations

```
relation CustomerOrder
  from Customer(id)
  to Order(customer_id)
```

```
bidirectional relation ProductCategory
  from Product(category_id)
  to Category(id)
```

```
relation OrderItem
  from Order(id)
  to Product(id) is

  quantity : Integer
  unit_price : Float
  discount : Float = 0.0
  line_total : Float
end
```

17.2.2 Relations avec aspects

```
relation CustomerOrder
  from Customer(id)
  to Order(customer_id) is

  created_at : Timestamp
  relationship_type : String

  aspects {
    DataLineage {
      upstream: "Customer",
      downstream: "Order",
      transformation: "direct_reference",
      business_rule: "Each order must belong to exactly one customer"
    }
  }
end
```

17.2.3 Type Relations (Templates réutilisables)

```
type relation OWNERSHIP is
  assigned_at : Timestamp
  role : String
  responsibilities : List of String
  review_frequency : Interval
end
```

```
relation CustomerDataOwnership of type OWNERSHIP is
  from Customer(id)
  to DataSteward(id)

  assigned_at: "2024-01-15T10:00:00Z"
  role: "data_steward"
  responsibilities: List["quality_monitoring", "access_control"]
  review_frequency: Interval 6 month
end
```

17.2.4 Règles sémantiques

- Une relation doit spécifier les entités source et cible
- Les champs de liaison (foreign keys) doivent exister dans les entités
- Les relations bidirectionnelles créent des liens dans les deux sens
- Les type relations permettent la réutilisation de patterns de relations
- Les aspects peuvent être appliqués aux relations pour la gouvernance

18 9

19 Traits

Les traits en Meshr-Lang sont un mécanisme de composition qui permet de définir des ensembles réutilisables de champs et de comportements. Ils favorisent la réutilisabilité et la cohérence dans la modélisation des données.

19.1 Concept et objectifs

19.1.1 Principe de composition

Les traits permettent de composer des entités en combinant des ensembles de champs prédéfinis, évitant ainsi la duplication de code et assurant la cohérence.

19.1.2 Avantages

- Réutilisabilité : Définir une fois, utiliser partout
- Cohérence : Garantir des structures uniformes
- Maintenabilité : Modifier un trait met à jour toutes les entités qui l'utilisent
- Modularité : Séparer les préoccupations en traits spécialisés

19.2 Syntaxe de base

19.2.1 Définition d'un trait

```
trait WithTimestamps is
  created_at : Timestamp
  updated_at : Timestamp
  created_by : String
  updated_by : String
end
```

19.2.2 Trait avec contraintes

```
trait WithContactInfo is
  email : String pattern "^[^@]+@[^@]+$"
  phone : String pattern "^\\+?[1-9]\\d{1,14}$"
  address : Address
end
```

19.2.3 Trait avec valeurs par défaut

```
trait WithStatus is
  status : String = "active"
  is_deleted : Boolean = false
  deleted_at : Timestamp
end
```

19.3 Utilisation des traits

19.3.1 Composition simple

```
entity Customer with WithTimestamps is
  id : String
  name : String
  email : String
end
```

19.3.2 Composition multiple

```
entity Employee with WithTimestamps, WithContactInfo, WithStatus is
  employee_id : String
  first_name : String
  last_name : String
  department_id : String
end
```

19.3.3 Composition avec aspects

```
entity Order with WithTimestamps, WithSecurity is
  id : String
  customer_id : String
  total_amount : Float

  aspects {
    DataClassification {
      level: "internal",
      steward: "orders@company.com"
    }
  }
end
```

19.4 Traits spécialisés

19.4.1 Traits de sécurité

```
trait WithSecurity is
  security_classification : String
  access_level : String
  encryption_required : Boolean = false
end
```

19.4.2 Traits de versioning

```
trait WithVersioning is
  version : String
  version_number : Integer
  is_current_version : Boolean = true
  previous_version_id : String
end
```

19.4.3 Traits de métadonnées

```
trait WithMetadata is
  tags : List of String = List[]
  metadata : Map of String to String = Map{}
  description : String
end
```

19.5 Règles de composition

19.5.1 Résolution des conflits

```
// En cas de conflit de noms, l'entité prévaut
trait TraitA is
  name : String
end

trait TraitB is
  name : String // Conflit potentiel
end

entity Customer with TraitA, TraitB is
  name : String // Résout le conflit explicitement
```

```
    id : String  
end
```

19.5.2 Héritage de contraintes

```
trait WithValidatedEmail is  
    email : String pattern "^[^@]+@[^@]+$"  
end
```

```
entity User with WithValidatedEmail is  
    id : String  
    // email hérite automatiquement du pattern  
end
```

19.5.3 Traits avec annotations

```
@Version("1.0.0")  
@Documented(summary="Common audit fields for all entities")  
trait WithAudit is  
    created_at : Timestamp  
    created_by : String  
    audit_trail : List of String = List[]  
end
```

21 Aspects

Les aspects en Meshr-Lang permettent d'attacher des métadonnées structurées aux entités, relations, et métriques. Ils sont essentiels pour implémenter la gouvernance des données, la sécurité, et la conformité.

21.1 Concept et objectifs

21.1.1 Principe de séparation des préoccupations

Les aspects séparent les préoccupations métier (structure des données) des préoccupations transversales (sécurité, gouvernance, qualité).

21.1.2 Domaines d'application

- Gouvernance des données : Stewardship, ownership, qualité
- Sécurité : Classification, chiffrement, contrôle d'accès
- Conformité : GDPR, SOX, HIPAA
- Observabilité : Métriques, monitoring, alerting
- Cycle de vie : Versioning, archivage, rétention

21.2 Syntaxe de base

21.2.1 Définition d'un aspect

```
aspect DataClassification is
  level : String // "public", "internal", "confidential", "restricted"
  encryption_required : Boolean
  access_control : String
  data_retention : Interval
  steward : String
end
```

21.2.2 Aspect avec contraintes

```
aspect GDPRCompliance is
  lawful_basis : String pattern "^(consent|contract|legal_obligation|vital_interest)"
  data_subject_rights : List of String
  retention_period : Interval
  deletion_date : Date
```

```

    consent_withdrawn : Boolean = false
    processing_purpose : String length 1..500
end

```

21.2.3 Aspect avec énumérations

```

enum ConfidentialityLevel is ("public", "internal", "confidential", "restricted")

```

```

aspect SecurityControls is
    confidentiality_level : ConfidentialityLevel
    integrity_required : Boolean = true
    availability_sla : Float range 0.0..1.0
    audit_logging : Boolean = true
    encryption_at_rest : Boolean = false
    encryption_in_transit : Boolean = true
end

```

21.3 Application des aspects

21.3.1 Sur les entités

```

entity Customer is
    id : String
    name : String
    email : String
    personal_data : String

```

```

aspects {
    DataClassification {
        level: "confidential",
        encryption_required: true,
        access_control: "role-based",
        data_retention: Interval 7 year,
        steward: "privacy@company.com"
    },
    GDPRCompliance {
        lawful_basis: "consent",
        data_subject_rights: List["access", "rectification", "erasure", "portability"]
        retention_period: Interval 7 year,
        processing_purpose: "Customer relationship management and service delivery"
    },

```

```

    SecurityControls {
      confidentiality_level: "confidential",
      availability_sla: 0.999,
      audit_logging: true,
      encryption_at_rest: true
    }
  }
end

```

21.3.2 Sur les relations

```

relation CustomerOrder
  from Customer(id)
  to Order(customer_id) is

  created_at : Timestamp

  aspects {
    DataLineage {
      upstream_entity: "Customer",
      downstream_entity: "Order",
      transformation_type: "direct_reference",
      business_rule: "Each order must belong to exactly one customer",
      impact_analysis: "Critical for order processing and customer analytics"
    }
  }
end

```

21.3.3 Sur les métriques

```

metric CustomerSatisfaction is
  source Customer
  calculation avg(satisfaction_score)

  aspects {
    MetricGovernance {
      business_owner: "customer.success@company.com",
      technical_owner: "analytics@company.com",
      refresh_frequency: Interval 1 day,
      data_freshness_sla: Interval 2 hour,

```

```

        accuracy_threshold: 95.0,
        business_criticality: "high"
    }
}
end

```

21.4 Aspects prédéfinis (StdLib)

21.4.1 Gouvernance des données

```

aspect DataStewardship is
    steward : String    // Email du data steward
    owner : String      // Email du data owner
    business_domain : String
    last_review : Date
    next_review : Date
    contact_email : String
end

```

```

aspect DataQuality is
    accuracy_threshold : Float range 0.0..1.0
    completeness_threshold : Float range 0.0..1.0
    consistency_rules : List of String
    validation_frequency : Interval
    quality_score : Float range 0.0..1.0
end

```

21.4.2 Cycle de vie des données

```

aspect DataLifecycle is
    created_at : Timestamp
    updated_at : Timestamp
    version : String
    status : DataStatus // active , deprecated , archived
    deprecation_date : Date
    archive_date : Date
    lifecycle_stage : LifecycleStage // development , staging , production
end

```

21.4.3 Observabilité et monitoring

```

aspect MonitoringConfig is
  alerts_enabled : Boolean = true
  monitoring_frequency : Interval = Interval 5 minute
  alert_thresholds : Map of String to Float
  dashboard_url : String
  runbook_url : String
end

```

21.5 Règles sémantiques

21.5.1 Application multiple

```

entity SensitiveData is
  id : String
  content : String

  aspects {
    // Plusieurs aspects peuvent être appliqués
    DataClassification { level: "restricted" },
    GDPRCompliance { lawful_basis: "legitimate_interests" },
    SecurityControls { confidentiality_level: "restricted" },
    DataLifecycle { status: "active" }
  }
end

```

21.5.2 Héritage via les traits

```

trait WithGDPR is
  gdpr_compliant : Boolean = true
end

entity PersonalData with WithGDPR is
  id : String
  personal_info : String

  aspects {
    GDPRCompliance {
      lawful_basis: "consent",
      data_subject_rights: List["access", "rectification", "erasure"]
    }
  }
}

```

end

21.5.3 Validation et cohérence

- Les aspects doivent être définis avant utilisation
- Les valeurs d'aspects doivent respecter les contraintes définies
- Les aspects peuvent référencer des énumérations et types personnalisés
- La cohérence entre aspects est vérifiée (ex: niveau de sécurité vs classification)

23 Métriques (Metrics)

23.1 Introduction

Les métriques constituent une fonctionnalité majeure de Meshr-Lang v0.2.0, permettant de définir des KPIs (Key Performance Indicators) et des mesures analytiques directement dans le modèle de données. Cette approche garantit la cohérence sémantique et la traçabilité entre les données source et les métriques dérivées.

23.1.1 Philosophie et objectifs

- **Cohérence sémantique** : Les métriques référencent directement les entités du domaine
- **Documentation vivante** : Les calculs sont documentés dans le modèle
- **Gouvernance intégrée** : Aspects de qualité et ownership directement attachés
- **Génération automatique** : SQL, dashboards, et APIs générés automatiquement

23.2 Syntaxe de base

23.2.1 Structure générale

```
metric MetricName is
  source EntityName
  calculation expression
  [aggregation expression]
  [unit "unit_string"]
  [outputs { field_definitions }]
  [dimensions { dimension_definitions }]
  [filters { filter_conditions }]
  [temporal { temporal_config }]
  [aspects { aspect_definitions }]
end
```

23.2.2 Exemple simple

```
metric TotalCustomers is
  source Customer
  calculation count(*)
  unit "customers"
end
```

23.2.3 Exemple complexe

```
metric MonthlyRevenue is
  source Order
  calculation sum(total_amount)
  unit "EUR"

  dimensions {
    month from date_trunc("month", order_date),
    region from related(Customer, customer_id).region
  }

  filters {
    status == "completed",
    order_date >= Date "2024-01-01"
  }

  temporal {
    window: Interval 1 month,
    refresh_frequency: Interval 1 day
  }
end
```

23.3 Composants détaillés

23.3.1 1. Source (obligatoire)

Spécifie l'entité source des données pour le calcul de la métrique.

```
// Source simple
metric CustomerCount is
  source Customer
  calculation count(*)
end

// Source avec jointure implicite
metric AverageOrderValue is
  source Order
  calculation avg(total_amount)
end
```

23.3.2 2. Calculation (obligatoire)

Définit l'expression de calcul principal de la métrique.

```
// Fonctions d'agrégation
metric TotalRevenue is
  source Order
  calculation sum(total_amount)
end

// Expressions arithmétiques
metric AverageMargin is
  source Product
  calculation avg((selling_price - cost_price) / selling_price * 100)
end

// Fonctions temporelles
metric DaysSinceLastOrder is
  source Customer
  calculation avg(current_date() - last_order_date)
end
```

23.3.3 3. Aggregation (optionnel)

Permet des calculs dérivés ou des agrégations multiples.

```
metric OrderStatistics is
  source Order
  calculation count(*)
  aggregation {
    total_orders: count(*),
    total_revenue: sum(total_amount),
    avg_order_value: avg(total_amount),
    max_order: max(total_amount)
  }
end
```

23.3.4 4. Unit (optionnel)

Spécifie l'unité de mesure de la métrique.

```
metric ProductWeight is
```

```

    source Product
    calculation avg(weight)
    unit "kg"
end

metric ResponseTime is
    source APICall
    calculation avg(response_time_ms)
    unit "ms"
end

```

23.3.5 5. Outputs (optionnel)

Définit les champs de sortie typés de la métrique.

```

metric CustomerAnalytics is
    source Customer
    calculation count(*)

    outputs {
        total_customers : Integer ,
        active_customers : Integer ,
        churn_rate : Float ,
        acquisition_rate : Float ,
        lifetime_value : Float
    }
end

```

23.3.6 6. Dimensions (optionnel)

Définit les axes d'analyse avec support du pattern matching.

```

metric SalesByRegion is
    source Order
    calculation sum(total_amount)

    dimensions {
        region from related(Customer, customer_id).region ,
        quarter from date_trunc("quarter", order_date) ,

        // Pattern matching sur les dimensions
    }
end

```

```

        customer_segment from match customer_type is
            "premium" -> "Premium",
            "standard" -> "Standard",
            "basic" -> "Basic"
        end
    }
end

```

23.3.7 7. Filters (optionnel)

Définit les conditions de filtrage des données.

```

metric ActiveCustomerRevenue is
    source Order
    calculation sum(total_amount)

    filters {
        status == "completed",
        related(Customer, customer_id).status == "active",
        order_date >= Date "2024-01-01",
        total_amount > 0.0
    }
end

```

23.3.8 8. Temporal (optionnel)

Configuration temporelle pour les métriques time-series.

```

metric DailyActiveUsers is
    source UserSession
    calculation count(distinct user_id)

    temporal {
        window: Interval 1 day,
        refresh_frequency: Interval 1 hour,
        historical_depth: Interval 2 year
    }
end

```

23.3.9 9. Aspects (optionnel)

Métadonnées de gouvernance et de qualité.

```

metric CriticalKPI is
  source Customer
  calculation count(*)

  aspects {
    MetricGovernance {
      business_owner: "ceo@company.com",
      technical_owner: "analytics@company.com",
      refresh_frequency: Interval 1 hour,
      data_freshness_sla: Interval 2 hour,
      accuracy_threshold: 99.0,
      business_criticality: "critical"
    }
  }
end

```

23.4 Pattern Matching avancé

23.4.1 Syntaxe du pattern matching

```

match expression is
  pattern1 -> result1 ,
  pattern2 -> result2 ,
  pattern3 or pattern4 -> result3 ,
  _ -> default_result
end

```

23.4.2 Patterns supportés

```

customer_tier from match annual_spend is
  amount >= 10000.0 -> "Gold",
  amount >= 5000.0 -> "Silver",
  amount >= 1000.0 -> "Bronze",
  _ -> "Standard"
end

```

```

age_group from match age is
  0..17 -> "Minor",
  18..64 -> "Adult",

```

```

    65..150 -> "Senior",
    _ -> "Unknown"
end

```

```

order_priority from match status is
    OrderStatus.urgent -> "High",
    OrderStatus.normal -> "Medium",
    OrderStatus.low_priority -> "Low"
end

```

```

business_day from match day_of_week is
    "monday" or "tuesday" or "wednesday" or "thursday" or "friday" -> true,
    "saturday" or "sunday" -> false
end

```

23.5 Exemple complet

```

@Version("1.0.0")
@author(name="Analytics Team", email="analytics@company.com")
@Documented(
    summary="Comprehensive customer analytics metric",
    description="Tracks customer behavior patterns with regional and temporal dimensions"
)
metric CustomerInsights is
    source Customer
    calculation count(*)

    aggregation {
        total_customers: count(*),
        active_customers: count(*) where status == "active",
        revenue_per_customer: avg(lifetime_value),
        churn_rate: count(*) where status == "churned" / count(*) * 100
    }

    unit "customers"

```

```

outputs {
  total_customers : Integer ,
  active_customers : Integer ,
  revenue_per_customer : Float ,
  churn_rate : Float
}

dimensions {
  region from address.country ,
  acquisition_quarter from date_trunc("quarter", created_at),

  customer_tier from match lifetime_value is
    amount >= 10000.0 -> "Premium",
    amount >= 5000.0 -> "Gold",
    amount >= 1000.0 -> "Silver",
    _ -> "Bronze"
  end,

  activity_level from match last_activity_days is
    0..7 -> "Very Active",
    8..30 -> "Active",
    31..90 -> "Moderate",
    91..365 -> "Low",
    _ -> "Inactive"
  end
}

filters {
  created_at >= Date "2024-01-01",
  status in List["active", "inactive"],
  not is_test_account
}

temporal {
  window: Interval 1 month,
  refresh_frequency: Interval 1 day,
  historical_depth: Interval 3 year
}

```

```

aspects {
  MetricGovernance {
    business_owner: "customer.success@company.com",
    technical_owner: "analytics@company.com",
    refresh_frequency: Interval 1 day,
    data_freshness_sla: Interval 4 hour,
    accuracy_threshold: 95.0,
    business_criticality: "high",
    dashboard_visibility: List["executive", "customer_success", "analytics"]
  },
  DataClassification {
    level: "internal",
    encryption_required: false,
    access_control: "role-based",
    data_retention: Interval 3 year,
    steward: "analytics@company.com"
  }
}
end

```

23.6 Règles sémantiques

23.6.1 Validation des références

- Les entités source doivent exister et être importées
- Les champs référencés doivent exister dans les entités
- Les fonctions d'agrégation doivent être compatibles avec les types de données

23.6.2 Cohérence temporelle

- Les configurations temporelles doivent être cohérentes
- refresh_frequency doit être $\leq$ window
- historical_depth doit être $\geq$ window

23.6.3 Types et contraintes

- Les expressions de calcul doivent retourner des types compatibles
- Les patterns de matching doivent couvrir tous les cas possibles
- Les filtres doivent retourner des valeurs booléennes

23.6.4 Génération d'artefacts

Les métriques Meshr-Lang génèrent automatiquement : - **Requêtes SQL** optimisées pour les bases de données - **Schémas dbt** pour les pipelines de transformation - **APIs REST** pour l'accès aux métriques - **Configurations Dataplex** pour la gouvernance - **Dashboards** avec visualisations appropriées

24 12

25 Modificateur sealed

Le modificateur sealed en Meshr-Lang permet de contrôler l'extensibilité des déclarations, particulièrement utile pour les énumérations et les hiérarchies de types où l'ensemble des valeurs possibles doit être fixe et connu à la compilation.

25.1 Concept et objectifs

25.1.1 Principe d'extensibilité contrôlée

Le modificateur sealed garantit qu'une déclaration ne peut pas être étendue ou modifiée en dehors de son module de définition, assurant ainsi la stabilité et la prédictibilité du modèle.

25.1.2 Cas d'usage

- Énumérations critiques pour la logique métier
- Hiérarchies de types fermées
- APIs stables nécessitant des garanties de compatibilité
- Domaines métier avec des valeurs strictement définies

25.2 Syntaxe

25.2.1 Énumération sealed

```
sealed enum PaymentStatus is ("pending", "completed", "failed", "refunded")
```

25.2.2 Trait sealed

```
sealed trait CoreEntity is
  id : String
  created_at : Timestamp
  updated_at : Timestamp
end
```

25.2.3 Entity sealed

```
sealed entity SystemConfiguration is
  config_key : String
  config_value : String
  is_active : Boolean
end
```

25.3 Règles sémantiques

25.3.1 Restrictions d'extension

- Une déclaration sealed ne peut pas être étendue dans d'autres modules
- Les valeurs d'une énumération sealed ne peuvent pas être ajoutées
- Les traits sealed ne peuvent pas être modifiés par composition

25.3.2 Garanties de stabilité

- Pattern matching exhaustif garanti sur les énumérations sealed
- APIs stables pour les intégrations externes
- Validation complète à la compilation

25.3.3 Exemples d'usage

```
// Cette énumération ne peut pas être étendue
sealed enum OrderStatus is (
  "draft",
  "submitted",
  "processing",
  "shipped",
  "delivered",
  "cancelled",
  "returned"
)

// Pattern matching exhaustif garanti
metric OrdersByStatus is
  source Order
  calculation count(*)

  dimensions {
    status_category from match status is
      "draft" or "submitted" -> "Pending",
      "processing" or "shipped" -> "In Progress",
      "delivered" -> "Completed",
      "cancelled" or "returned" -> "Cancelled"
  }
end
```

```
// Trait de base qui ne peut pas être modifié
sealed trait AuditableEntity is
  created_at : Timestamp
  created_by : String
  updated_at : Timestamp
  updated_by : String
  version : Integer
end

// Utilisation dans les entités
entity Customer with AuditableEntity is
  id : String
  name : String
  email : String
end
```

27 Artefacts définissables

Meshr-Lang permet de définir différents types d'artefacts pour modéliser complètement une architecture Data Mesh. Chaque artefact a un rôle spécifique dans l'écosystème des données.

27.1 Types d'artefacts supportés

27.1.1 Artefacts de structure

- **Modules** : Unités d'organisation et d'encapsulation
- **Entités** : Objets métier centraux avec identité
- **Records** : Structures de données légères
- **Énumérations** : Ensembles finis de valeurs

27.1.2 Artefacts de composition

- **Traits** : Ensembles réutilisables de champs et comportements
- **Aspects** : Métadonnées transversales (gouvernance, sécurité)
- **Relations** : Connexions sémantiques entre entités
- **Type Relations** : Templates de relations réutilisables

27.1.3 Artefacts de métadonnées

- **Annotations** : Métadonnées attachables aux déclarations
- **Types personnalisés** : Extensions du système de types

27.1.4 Artefacts analytiques

- **Métriques** : KPIs et mesures avec pattern matching
- **Dimensions** : Axes d'analyse pour les métriques
- **Filtres** : Conditions de sélection des données

27.2 Règles de définition

27.2.1 Nommage et conventions

- **PascalCase** pour les entités, traits, aspects : CustomerOrder, WithTimestamps
- **camelCase** pour les champs et variables : customerId, createdAt
- **SCREAMING_SNAKE_CASE** pour les constantes : MAX_RETRY_COUNT
- **snake_case** pour les valeurs d'énumération : pending_approval

27.2.2 Ordre de déclaration

1. Annotations du module
2. Déclaration du module
3. Imports
4. Exports
5. Types et énumérations
6. Annotations personnalisées
7. Traits et aspects
8. Entités
9. Relations
10. Métriques

27.2.3 Visibilité et encapsulation

```
// Export public - visible par tous les modules
export { Customer, Order }

// Export interne - visible uniquement dans le domaine
@Internal
export { InternalMetrics }

// Déclaration privée - visible uniquement dans le module
entity PrivateConfig is
  // ...
end
```

27.3 Exemples d'architecture complète

27.3.1 Module de domaine métier

```
@Version("2.0.0")
@author(name="E-commerce Team", email="ecommerce@company.com")
@Documented(summary="E-commerce order management domain")
module ecommerce.orders

import { Customer } from ecommerce.customers
import { Product } from ecommerce.catalog
import { WithTimestamps, WithSecurity } from meshr.core
import { DataClassification } from meshr.security
```

```

export { Order, OrderItem, OrderStatus, OrderMetrics }

// Énumération sealed pour garantir l'exhaustivité
sealed enum OrderStatus is (
  "draft", "submitted", "processing",
  "shipped", "delivered", "cancelled"
)

// Trait réutilisable
trait WithOrderInfo is
  order_number : String pattern "^ORD-\\d{8}$"
  total_amount : Float range 0.01..999999.99
  currency : String length 3..3
end

// Entité principale
entity Order with WithTimestamps, WithSecurity, WithOrderInfo is
  id : String
  customer_id : String
  status : OrderStatus
  shipping_address : Address
  billing_address : Address

  aspects {
    DataClassification {
      level: "internal",
      steward: "ecommerce@company.com",
      data_retention: Interval 7 year
    }
  }
end

// Entité liée
entity OrderItem is
  id : String
  order_id : String
  product_id : String
  quantity : Integer range 1..1000
  unit_price : Float
  line_total : Float

```

end

// Relations

relation OrderCustomer from Order(customer_id) to Customer(id)

relation OrderItems from Order(id) to OrderItem(order_id)

relation ItemProduct from OrderItem(product_id) to Product(id)

// Métrique analytique

metric OrderMetrics is

source Order

calculation count(*)

dimensions {

status_group from match status is

"draft" or "submitted" -> "Pending",

"processing" or "shipped" -> "In Progress",

"delivered" -> "Completed",

_ -> "Other"

end,

month from date_trunc("month", created_at)

}

filters {

created_at >= Date "2024-01-01",

total_amount > 0.0

}

aspects {

MetricGovernance {

business_owner: "ecommerce.manager@company.com",

refresh_frequency: Interval 1 hour,

business_criticality: "high"

}

}

end

Cette architecture complète démontre l'utilisation coordonnée de tous les artefacts Meshr-Lang pour modéliser un domaine métier riche et gouverné.

28 Annexes

28.1 Annexe A — Grammaire EBNF (référence)

La grammaire EBNF (Extended Backus-Naur Form) de Meshr-Lang définit formellement la syntaxe du langage. Cette annexe présente les règles de production principales.

Note : Cette grammaire est maintenue dans le fichier `grammar/meshr_module.ebnf` du projet. La version complète et à jour se trouve dans ce fichier source.

28.1.1 Structure générale

```
module = module_annotation* "module" module_name
        import_declaration*
        export_declaration*
        declaration*
```

```
module_name = identifier ( "." identifier )*
```

```
declaration = enum_declaration
             | record_declaration
             | trait_declaration
             | aspect_declaration
             | annotation_declaration
             | entity_declaration
             | relation_declaration
             | type_relation_declaration
             | metric_declaration
```

28.1.2 Métriques (v0.2.0)

```
metric_declaration = metric_annotation* "metric" identifier "is"
                   metric_component*
                   "end"
```

```
metric_component = source_component
                  | calculation_component
                  | aggregation_component
                  | unit_component
                  | outputs_component
                  | dimensions_component
                  | filters_component
```

```

        | temporal_component
        | aspects_component

match_expression = "match" expression "is "
                  pattern_case+
                  "end"

pattern_case = pattern ("or" pattern)* "->" expression ", "?

pattern = literal_pattern
        | range_pattern
        | enum_pattern
        | wildcard_pattern

```

28.1.3 Types et contraintes

```

type_declaration = "type" identifier "is" type_definition constraint*

constraint = pattern_constraint
          | range_constraint
          | length_constraint
          | precision_constraint

field_declaration = identifier ":" type_reference constraint* default_value?

```

Référence complète : La grammaire EBNF complète avec toutes les règles de production est disponible dans `grammar/meshr_module.ebnf` (262 lignes, version 0.2.0).

28.2 Annexe B — Grammaire ANTLR (référence, Python target)

La grammaire ANTLR de Meshr-Lang est utilisée pour l'implémentation du parser et du Language Server Protocol (LSP).

Note : Cette grammaire est maintenue dans le fichier `grammar/MeshrModule.g4` du projet. La version complète et à jour (801 lignes, version 0.2.0) se trouve dans ce fichier source avec support complet des métriques et du pattern matching.

28.2.1 Tokens principaux (v0.2.0)

```

// Mots-clés de base
MODULE: 'module';
IMPORT: 'import';

```

```

EXPORT: 'export';
ENTITY: 'entity';
ENUM: 'enum';
TRAIT: 'trait';
ASPECT: 'aspect';

// Mots-clés métriques (nouveau v0.2.0)
METRIC: 'metric';
SOURCE: 'source';
CALCULATION: 'calculation';
AGGREGATION: 'aggregation';
DIMENSIONS: 'dimensions';
FILTERS: 'filters';
TEMPORAL: 'temporal';
MATCH: 'match';

```

28.2.2 Règles de parsing

```

module: moduleAnnotation* MODULE moduleName
      importDeclaration*
      exportDeclaration*
      declaration* EOF;

metricDeclaration: metricAnnotation* METRIC IDENTIFIER IS
                  metricComponent*
                  END;

matchExpression: MATCH expression IS
                patternCase+
                END;

```

Référence complète : La grammaire ANTLR complète avec toutes les règles de parsing est disponible dans `grammar/MeshrModule.g4` avec génération automatique du lexer, parser, et visitor pour Python.

Version du document : 0.2.0 - Septembre 2025

Dernière mise à jour : 2025-09-18

Support : Métriques complètes et pattern matching